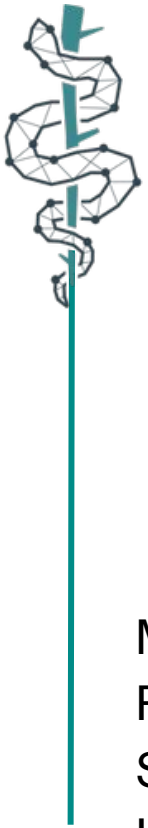


OVERVIEW OF MACHINE LEARNING ALGORITHMS

Mahdieh Mallahnezhad
Research Assistant, MSc, Computer Science
University of Alberta

Outline of Today’s Lecture

🔁 Review Of Essential Terms From Lecture 1

📊 Ml Workflow: Problem → Data → Models → Validation

🤖 Core Machine Learning Algorithms (Pre–Deep Learning)

❓ Interactive Menti Questions Throughout

Review: AI vs ML, What's the Difference?

AI = the broad goal of simulating human intelligence

ML = a subset of AI that learns from data (vs fixed rules)

🩺 When do we use ML in healthcare?

- When patterns are too complex for humans to write rules
- When we have data with examples (patient records with known outcomes)

📱 Where is ML in healthcare?

(Talk to your neighbour)

💭 Think of one real example you've seen or heard about ML in healthcare.

Use anything: apps, hospital systems, medical imaging, devices, etc.

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

Review: Where Does Deep Learning Fit?

- ❏ Data is large and complex
- ❏ Patterns are too rich for classical ML
- ❏ Inputs are images, signals, or language
- ❏ Relationships are nonlinear and high-dimensional

🩺In healthcare:

- ❏ Detect cancer from images
- ❏ Read ECGs automatically
- ❏ Summarize clinical notes
- ❏ Predict diseases from raw signals

Review: Types of ML (how the model learns)

- **Supervised Learning**

  Model learns from labeled data (input → known output)

  🔍 Example: Predicting diabetes based on lab results

- **Unsupervised Learning**

  Model finds structure in unlabeled data

  🔍 Example: Clustering patients with similar symptoms into subtypes

- **Reinforcement Learning**

  Agent learns by trial and error, with feedback

  🔍 Example: A rehabilitation robot that adapts support levels based on patient response

Review: Common Machine Learning Tasks

Supervised learning

📊 Classification: Predict discrete categories

"Will this patient develop sepsis?" (yes/no)

📈 Regression: Predict continuous values

"What is this patient's 30-day mortality risk?" (0-100%)

Unsupervised learning

🔵🔴🟢Clustering: Group similar data points

“Identifying patient subgroups that respond differently to treatment”

🗜️Dimensionality Reduction: Reduce feature space

“Reducing 20,000 gene expressions to key patterns”

Review: Types of Data in Healthcare

📄 Structured data: EMRs, lab values, vitals

🖼️ Images: X-rays, MRIs

📈 Time series: ECGs, wearable sensors

📝 Unstructured text: clinical notes, discharge summaries

🤖 Machine learning models consume features extracted from this data

Review: Applications of ML in Healthcare

- 🩺 Diagnosis (e.g., cancer)
- 🔮 Prognosis prediction (e.g., mortality risk, readmission)
- 💊 Drug discovery (e.g., compound screening)
- 🧬 Personalized medicine (e.g., stratifying by response)
- 🧠 Clinical decision support (e.g., sepsis alerts)
- 🖼️ Medical imaging (e.g., pneumonia detection from X-ray)
- 🤖 Robotics & chatbots (surgery assistance, patient triage)

The ML Workflow: How Does It All Come Together?

- Define the question and the goal
- Collect & prepare data
- Choose appropriate features
- Choose an algorithm
- Evaluate the model
- Deploy and monitor it in the real world

Step 1:

Define the question and the goal

What Makes a Problem “Learnable”

Before starting an ML project, verify you have:

✓ Clear clinical goal: Can you state it as predict/classify/discover/rank?
✓ Relevant, available inputs: Do you have access to meaningful features (labs, vitals, imaging)?
✓ Defined outcome: Is the target measurable and clearly defined?
✓ Sufficient data: Do you have hundreds to thousands of examples?

💡 **Clinical goal determines the ML task type:**

Predict category → Classification
Predict value → Regression
Find patterns → Clustering

Evaluating an ML Problem Example

✅ Good example:

"Given a patient's first 6 hours of ICU data, can we predict if they will develop sepsis in the next 24 hours?"

✓ Clear goal (predict binary outcome)
✓ Available inputs (ICU vitals, labs routinely collected)
✓ Defined outcome (clinical diagnostic criteria)
✓ Sufficient data (hospitals have many ICU cases)

❌ Poor example:

"Why do patients feel better on weekends?"

✗ Seeks explanation rather than prediction
✗ "Feel better" is vague and subjective
✗ Outcome not measurable
✗ No clear input features or timeframe

📱 Is this a Good ML Question?

(Talk to your neighbour)

1. **“Why do patients prefer Dr. X over Dr. Y?”**
2. **“Can we group diabetic patients into subgroups based on their lab trends?”**

💭 For each, answer:

- **Is this suitable for ML (yes/no)?**
- **If yes, is it classification, regression, or clustering?**

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

Step 2:

Collect and Prepare Data

What Does Our Data Look Like?

Supervised learning

- Patient information → features
- A known outcome → the label

Predicting if a patient has the flu:

- Features: fever, sore throat, runny nose, cough, body aches
- Label: flu test positive (yes/no)

Unsupervised learning

- Patient information → features
- No label

Grouping patients with common cold symptoms:

- Features: congestion, sneezing, headache, fatigue
- Patterns:
    - ❑ “mild cold” group
    - ❑ “allergy-like cold” group
    - ❑ “cold with fever” group

Step 3:

Choose appropriate features

What information should the model use?

- Available before the outcome happens
- Clinically relevant
- Reliable and consistently recorded

Predict whether a patient with cold-like symptoms has the flu

✔️ Fever
(that's the label)
✔️ Sore throat
✔️ Cough
✔️ Body aches

❌ Flu test result

❌ Appointment time
❌ Postal code

📱 Discuss: Which Features Are Fair?

(Talk to your neighbour)

💭 **We want to predict who will miss their physiotherapy appointment (no-show).**

- ❑ Age
- ❑ Previous missed appointments
- ❑ Reason for physiotherapy (post-surgery, sports injury, etc.)
- ❑ Distance from clinic
- ❑ Whether the patient has already been labelled “difficult” in notes
- ❑ Clinic name
- ❑ Final recovery outcome (improved / not improved)

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

Step 4:

Choose a machine learning algorithm

Before We Begin: The Algorithms We'll Cover

Supervised learning (when we have labels):

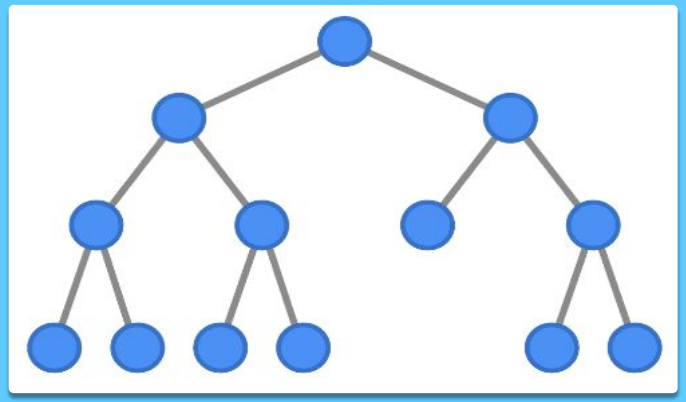
- Decision Trees: intuitive, rule-based
- Random Forests: ensemble of trees
- Logistic Regression: probabilities & risk scores
- K-Nearest Neighbors (KNN): similarity-based
- Support Vector Machines (SVM): separating boundaries

Unsupervised learning (no labels):

- K-Means Clustering: finding natural patient subgroups
- PCA (Principal Component Analysis): reducing many features into key patterns

DECISION TREE

Classification/ Regression



- Splits data based on yes/no questions
- Continues until reaching pure groups
- Makes prediction based on majority class

🎯 The Core Idea: Playing "20 Questions" to Diagnose

An experienced doctor diagnoses a patient:

- Don't randomly test everything
  - Wastes time, money, and doesn't narrow down the diagnosis
- Ask strategic yes/no questions in order
  - "Chest pain?" reveals more than "Eye color?"
- Each answer narrows down the possibilities
  - YES to chest pain → cardiac issues; NO → other causes
- Stop when the answer becomes clear
  - Confident in diagnosis → stop asking questions

A Decision Tree does exactly this: it learns the best questions to ask and in what order!

📱 Let's Choose the Best Question to Ask!

Scenario: You're diagnosing patients in the ER

You have 100 patients:

- 45 have bacterial pneumonia
- 55 are healthy (or minor viral infections)

Your decision tree needs to ask ONE question first.

Goal: Separate pneumonia patients from healthy patients as clearly as possible

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📊 Let's Test Each Question

A) Fever present? (Groups still mixed - not ideal)

- ├─ YES: 40 pneumonia, 30 healthy (57% pneumonia) 😕
- └─ NO: 5 pneumonia, 25 healthy (17% pneumonia)

B) Oxygen saturation < 92%? ⭐ (Very pure groups! Best first split!)

- ├─ YES: 42 pneumonia, 8 healthy (84% pneumonia) 🎯
- └─ NO: 3 pneumonia, 47 healthy (6% pneumonia) 🎯

C) Productive cough? (Similar to fever - groups mixed)

- ├─ YES: 38 pneumonia, 28 healthy (58% pneumonia) 😕
- └─ NO: 7 pneumonia, 27 healthy (21% pneumonia)

D) Age > 65? (Not very helpful)

- ├─ YES: 20 pneumonia, 15 healthy (57% pneumonia) 😕
- └─ NO: 25 pneumonia, 40 healthy (38% pneumonia)

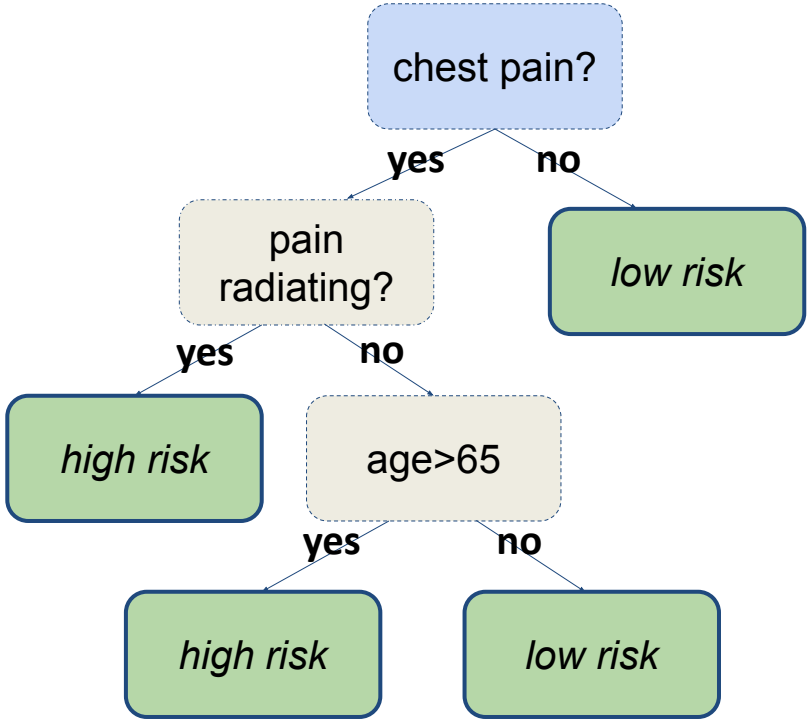
How a Decision Tree Works (classification)

Features you collected:

- Chest pain (Yes/No)
- Pain radiating (Yes/No)
- Age

Label (what we want to predict):

- Heart attack (Yes/No)



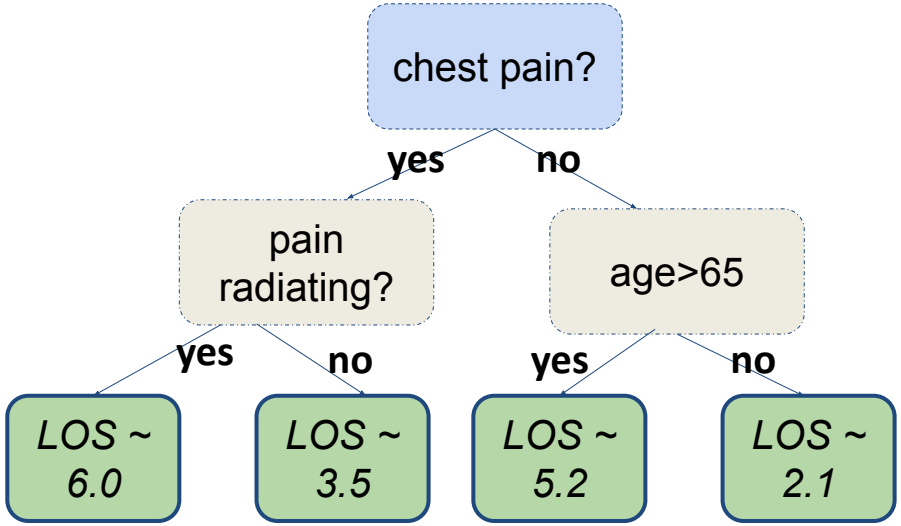
How a Decision Tree Works (regression)

Features you collected:

- Chest pain (Yes/No)
- Pain radiating (Yes/No)
- Age

Label (what we want to predict):

- **This is not Yes/No**
- Length of Stay→ Regression!



When to use a decision tree

Strengths

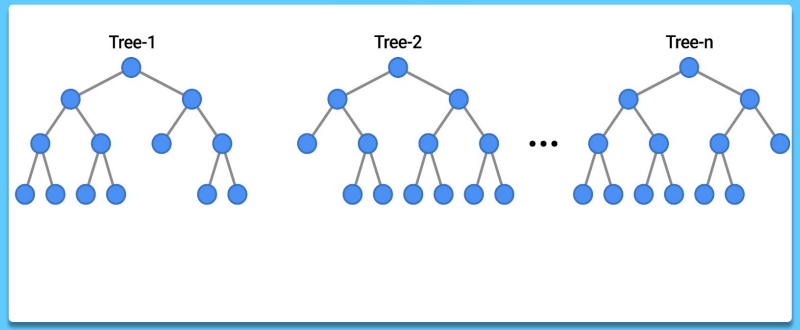
- Highly interpretable: mirrors clinical decision flowcharts
- Works with mixed data (numbers + yes/no + categories)
- Captures non-linear rules (conditions that only matter in combination)
- Quick to train, good baseline model

Weaknesses

- Prone to overfitting if the tree grows too deep
- Unstable: small data changes → different tree
- Struggles with subtle patterns spread across many variables
- Usually weaker than ensemble methods (Random Forest, XGBoost)

RANDOM FOREST

Classification/ Regression



- Builds many decision trees (100s or 1000s)
- Each tree sees slightly different training data
- Each tree votes on the prediction
- Final answer = majority vote (or average for regression)

🎯 The Core Idea: Consulting Multiple Doctors

You wouldn't trust just one doctor's opinion for a serious diagnosis, right?

- Get multiple expert opinions
  - 5 different doctors examine the patient independently
- Each doctor has slightly different experience
  - Doctor A specializes in imaging; Doctor B focuses on lab work; Doctor C relies on patient history
- Each makes their own recommendation
  - Doctor A: "Surgery"; Doctor B: "Surgery"; Doctor C: "Medication"; Doctor D: "Surgery"; Doctor E: "Surgery"
- Take a vote for final decision
  - 4 out of 5 say surgery → Final decision: Surgery ✓

A Random Forest does exactly this: builds many "expert" decision trees and takes a vote!

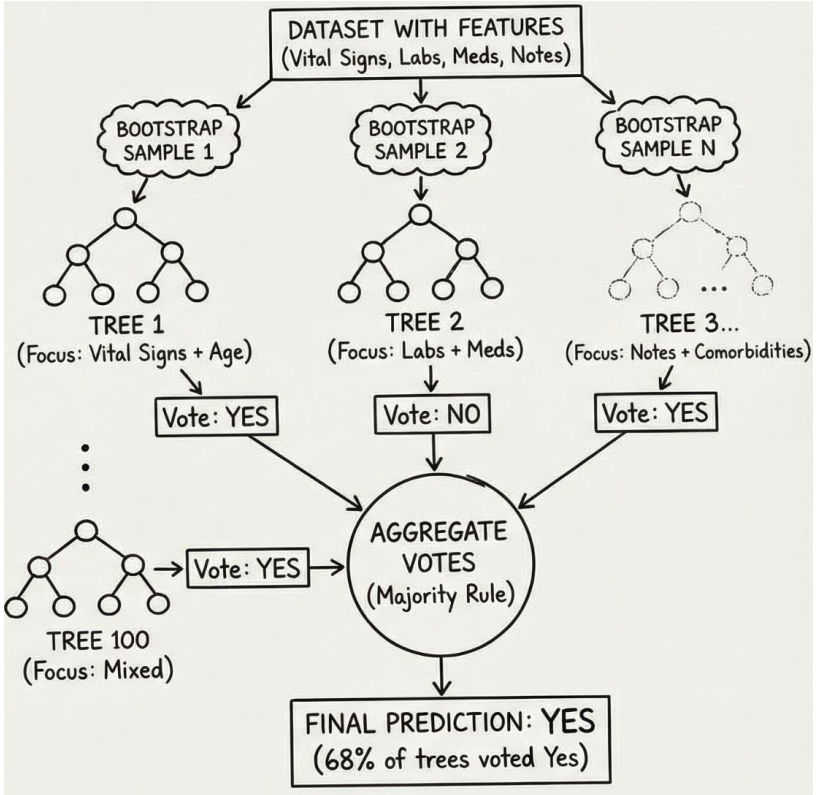
How a Random Forest Works

The setup:

- Goal: Predict if a patient will be readmitted to ICU within 48 hours
- Features: Vital signs, lab values, medications, nursing notes sentiment

Label (what we want to predict):

- Readmitted (Yes/No)



When to use a random forest

Strengths

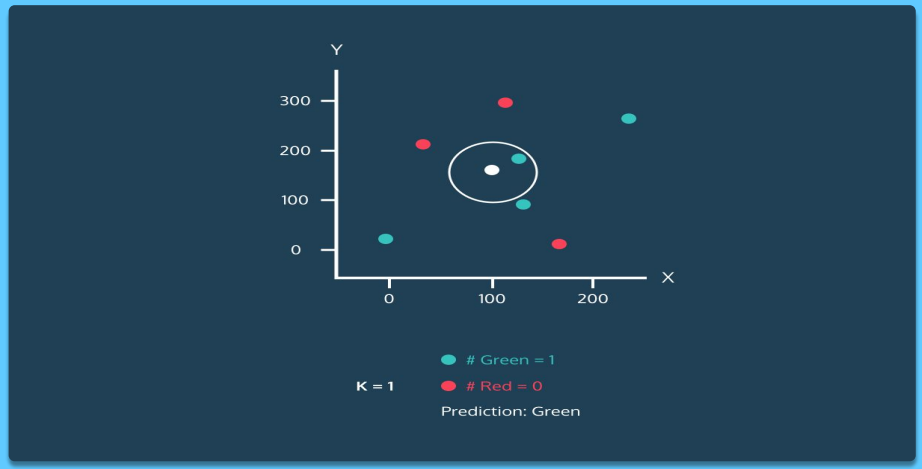
- Much more accurate than single decision trees
- Handles missing data well
- Provides feature importance rankings
- Works with mixed data types

Weaknesses

- Black box" - harder to explain than single tree
- Slower to train and predict
- Requires more memory
- Less intuitive for clinical staff

K Nearest Neighbors (KNN)

Classification/ Regression



- Store all training examples in memory
- For new patient, find K most similar patients
- Measure similarity using distance in feature space
- Predict most common label among neighbors (or average for regression)

🎯 The Core Idea: Finding Similar Past Cases

Want to diagnose a new patient? Look at similar past patients!

- Find patients who "look like" this one
  - Similar lab values, vitals, symptoms
- See what happened to those similar patients
  - Did they have iron deficiency? Thalassemia? B12 deficiency?
- Take a vote among the neighbors
  - If 6 out of 7 similar patients had iron deficiency → probably iron deficiency
- No complex training needed!
  - Just store past patients and search for matches

KNN is like asking: "Show me patients who looked just like this one. What was their diagnosis?"

📱 How Would You Diagnose This Patient?

You're diagnosing a patient with anemia. You have their lab results:

- Hemoglobin: 9.2 g/dL (low)
- MCV: 72 fL (low)
- Ferritin: 8 ng/mL (low)
- TIBC: 420 µg/dL (high)

You have a database of 1,000 past patients with known diagnoses.

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📊 Let's Test Each Option

A) ONE patient (K=1):

"Patient #347 had Hgb=9.1, MCV=71 → Iron deficiency"

→ What if #347 was misdiagnosed? Or unusual? → Too risky!

B) 5-7 similar patients (K=5-7): ⭐

Find 7 patients with closest lab values:

- 6 had Iron deficiency
- 1 had Thalassemia

→ Strong consensus! Likely iron deficiency.

→ Even if 1-2 are wrong, majority is still correct!

C) ALL patients (K=1000):

Includes patients with very different labs

→ Not helpful!

How KNN works

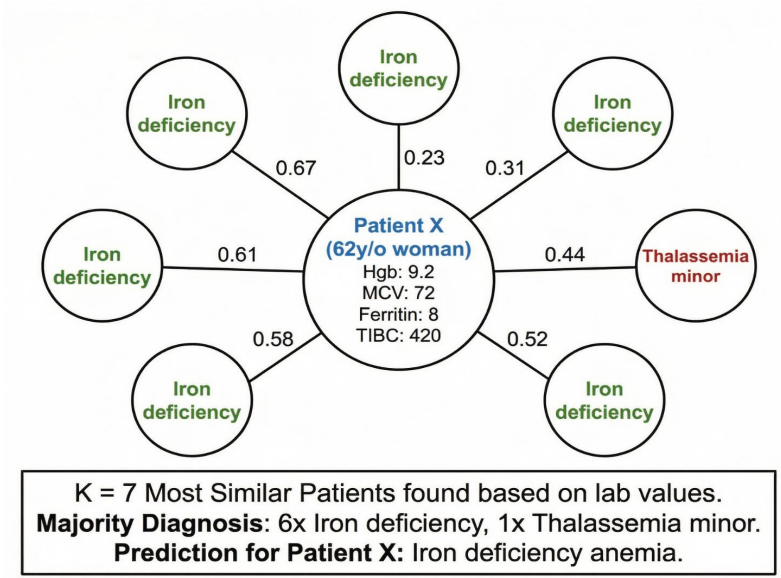
Features you collected:

- Hemoglobin (g/dL)
- Mean Corpuscular Volume - MCV (fL)
- Ferritin (ng/mL)
- TIBC - Total Iron Binding Capacity (µg/dL)

Label (what we want to predict):

- Iron deficiency anemia
- Thalassemia minor
- Anemia of chronic disease
- B12/Folate deficiency

Distance	Hgb	MCV	Ferritin	TIBC	Diagnosis
0.23	9.0	70	6	430	Iron deficiency ✓
0.31	9.5	73	10	410	Iron deficiency ✓
0.44	8.8	68	5	445	Iron deficiency ✓
0.52	9.1	74	12	400	Thalassemia minor
0.58	9.3	71	7	425	Iron deficiency ✓
0.61	8.9	69	9	415	Iron deficiency ✓
0.67	9.4	72	11	405	Iron deficiency ✓



How Many Neighbors Should We Ask?

K = 1 (Too Small) ❌

Prediction: Based on single nearest patient
Problem: Very sensitive to noise and outliers
Example: That one blue patient might be a misdiagnosis
Result: Overfitting - high variance, unstable

K = 50 (Too Large) ❌

Prediction: Based on 50 patients (including very distant ones)
Problem: Includes patients who aren't actually similar
Example: Averaging with very different patients
Result: Underfitting - misses local patterns

When to use KNN

Strengths

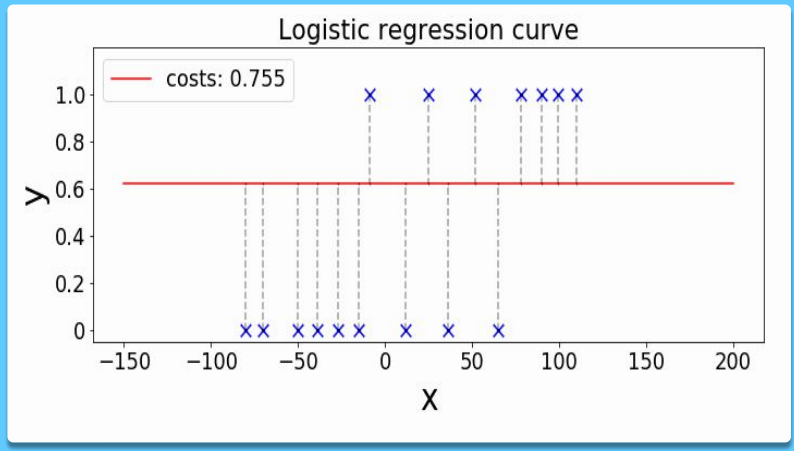
- Very intuitive: “patients similar to this patient”
- No training required (lazy learner)
- Works well for small datasets
- Good for problems where similarity = meaning
- Can handle multi-class problems naturally

Weaknesses

- Slow for large datasets (must compare to all past patients)
- Sensitive to irrelevant features
- Struggles in high dimensions (too much noise)
- Hard to explain why certain neighbors were close

Logistic regression

Classification



- Outputs a probability (0–1)
- Draws the best separating line between classes
- Converts probability → Yes/No using a threshold
- Great for risk scoring models

🎯The Core Idea: Building an Automatic Risk Calculator

You've seen risk calculators in medicine - they give you a percentage:

- Doctor fills in: Age, Blood Pressure, Cholesterol, Smoking
  - Each factor adds to the risk
- Calculator outputs: "This patient has 27% risk"
  - Not just yes/no - an actual percentage!
- Someone decided how much each factor matters
  - "Age adds 2 points, smoking adds 5 points"
- But who decided those numbers?
  - Usually expert doctors created the formula by hand

Logistic Regression does this automatically - it learns the formula from data!

📱 Building a Risk Score

Scenario: You want to predict heart attack risk using:

- Age
- Exercise
- Blood pressure
- Smoking status

Goal: Understand how Logistic Regression assigns importance to different risk factors

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📊 Different Factors, Different Importance!

Logistic Regression learns the optimal WEIGHT for each factor!

Clinically we know:

- Smoking: VERY strong predictor
- Age: Strong predictor
- High BP: Moderate predictor
- Exercise: Protective

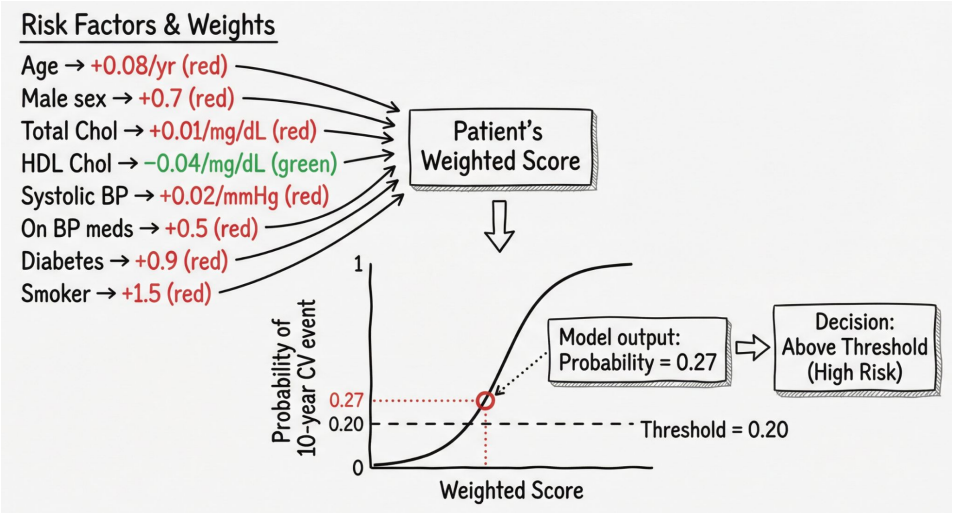
How Logistic Regression Works

Features you collected:

- Age: 62 years
- Sex: Male
- Total cholesterol: 240 mg/dL
- HDL cholesterol: 35 mg/dL
- Systolic BP: 152 mmHg
- On BP medication: Yes
- Diabetes: No
- Smoker: Yes

Label (what we want to predict):

- Cardiovascular event in next 10 years



How Do We Choose the Threshold?

Threshold = 0.20 (Too Low) ❌

Prediction: Flags many patients as high-risk
Problem: Too many false positives
Example: Many low-risk patients trigger alerts → alarm fatigue
Result: Over-calling risk, unnecessary tests, low precision

Threshold = 0.80 (Too High) ❌

Prediction: Flags only very high-probability cases
Problem: Many true cases are missed
Example: Early patients fall below the threshold → no alert
Result: Under-calling risk, dangerous false negatives

When to use Logistic regression

Strengths

- Produces probabilities medical staff can interpret
- Lightweight, extremely fast
- Few parameters → works well even with limited data
- Good baseline model for risk prediction

Weaknesses

- Only captures linear relationships unless features are engineered
- Poor at complex interactions (unlike trees / RF)
- Sensitive to unscaled features (needs preprocessing)
- Can be misled by outliers

Support Vector Machine (SVM)

Classification/ Regression

- Finds the best separating boundary
- Maximizes margin between classes
- Support vectors are key data points
- Can handle non-linear boundaries

🎯The Core Idea: Inflating a Balloon Between Two Groups

Imagine separating two groups of patients on a chart:

- Healthy patients on the left / Sick patients on the right
- Don't want a boundary that BARELY separates
- New patient, slightly different labs → prediction flips

Solution:

- Inflate a balloon between groups
- Keeps expanding until touches closest patients on both sides
- Patients it touches = "Support Vectors"
- These edge patients define the boundary

SVM does exactly this: inflates the boundary to maximum width for the most stable predictions!

📱 Where Should We Draw the Line?

Scenario:
You're separating healthy patients from sick patients based on lab values.

- You can draw the boundary line anywhere between the two groups.
- Where should you draw it?

Goal: Understand why SVM maximizes the margin

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📊 Let's Test the Options

A) Close to healthy patients ❌

Problem: New healthy patient slightly off → misclassified as sick!

Unstable boundary - no room for variation

B) Close to sick patients ❌

Problem: New sick patient slightly off → misclassified as healthy!

Unstable boundary - no room for variation

C) Middle with maximum space ✓ ⭐

✓ Far from both groups

✓ Room for natural variation in lab values

✓ Stable - small changes won't flip predictions

→ This is what SVM does!

How SVM works

Features you collected:

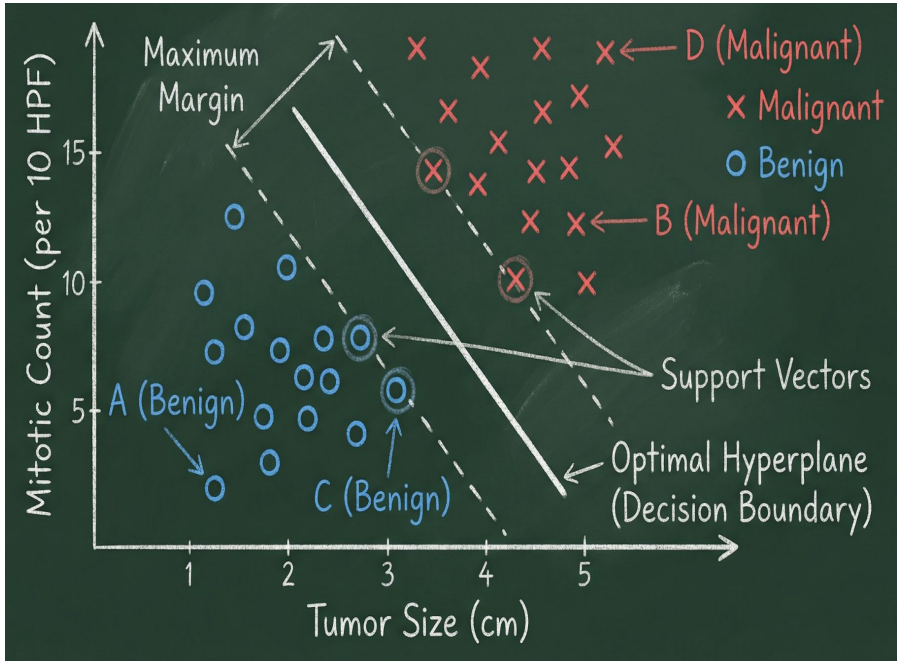
- Tumor size (cm)
- Mitotic count (per 10 HPF)

Label (what we want to predict):

- Benign vs Malignant tumor

For patients with known diagnosis:

- A: 1.2 cm, 2 mitoses → Benign
- B: 4.5 cm, 12 mitoses → Malignant
- C: 2.0 cm, 3 mitoses → Benign
- D: 5.1 cm, 15 mitoses → Malignant
- ... (100s more patients)



When to use SVM

Strengths

- Effective in high-dimensional spaces (many features)
- Memory efficient: only stores support vectors
- Works well when classes are clearly separated
- Resistant to overfitting

Weaknesses

- Slow on large datasets (training time grows quickly)
- Doesn't directly give probabilities (needs extra work)
- Hard to interpret: "why did it decide this?"
- Sensitive to feature scaling (must normalize data)

📱 Which Algorithm Would You Start With?

💭**For each scenario, choose the algorithm you'd try first:**

1. Quick bedside risk score
   - ❏ You want a simple risk score for post-operative bleeding that surgeons can understand and maybe even calculate on paper.

2. Many variables, modest interpretability
   - ❏ You have lots of lab values and vitals and just want a strong model to predict ICU transfer in the next 12 hours; you're OK with a more complex model.

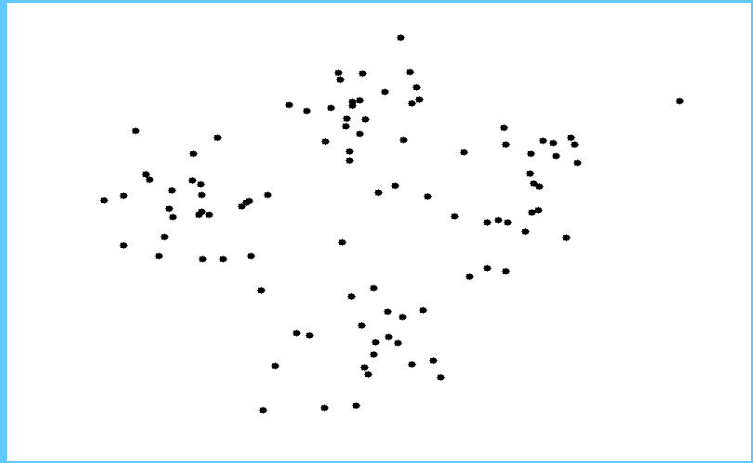
3. Small dataset, maybe nonlinear boundary
   - ❏ You have only a few hundred patients and suspect the boundary between "complication" vs "no complication" is not a straight line.

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

K-Means Clustering

Unsupervised Learning

No Labels Needed!



- Groups similar patients together
- You specify K (number of clusters)
- Discovers natural subgroups in data
- No predefined categories needed

🎯The Core Idea: Finding Patient Subgroups

Imagine you have 100 diabetic patients:

- They all have diabetes, but respond differently to treatment
  - Some do well, others struggle
- You suspect there are subgroups
  - But you don't know what they are or how many
- Group patients by similarity in their data
  - Blood sugar patterns, BMI, medication response
- Natural groups emerge
  - "Well-controlled", "Medication-resistant", "Newly-diagnosed"

K-Means does exactly this: discovers hidden patient subgroups!

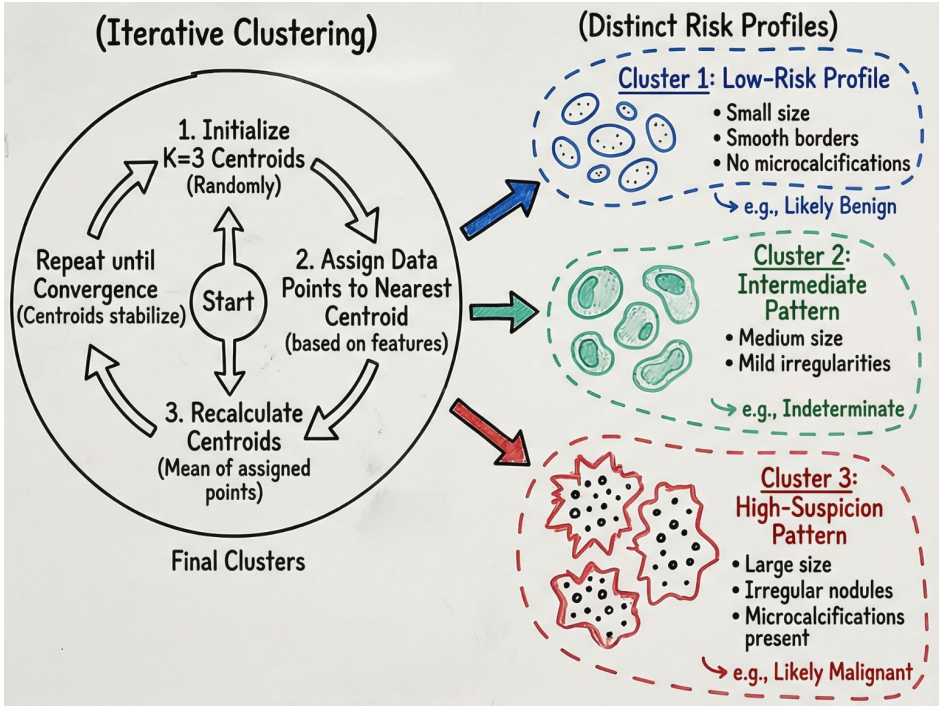
How K-Means Works

Ultrasound features of thyroid nodules:

- Nodule size
- Echogenicity
- Vascularity score
- Border irregularity
- Microcalcifications (yes/no)

No labels

- We do NOT tell the model which nodules are benign or malignant.



How Many Clusters Should We Use?

K = 2 (Too Few Clusters) ❌

Prediction: Mixes very different patients into broad groups
Problem: Low-risk and high-risk patterns get combined
Example: Thyroid nodules all fall into “benign-ish” or “suspicious-ish”
Result: Underfitting, loses meaningful subtypes

K = 8 (Too Many Clusters) ❌

Prediction: Creates tiny micro-groups
Problem: Hard to interpret clinically
Example: Slight variations in size or texture become separate clusters
Result: Overfitting — noisy, fragmented, no clear clinical value

When to use K-Means Clustering

Strengths

- Simple and intuitive concept
- Fast on large datasets
- Scales well to many features
- Discovers hidden subgroups without labels
- Great for exploratory analysis

Weaknesses

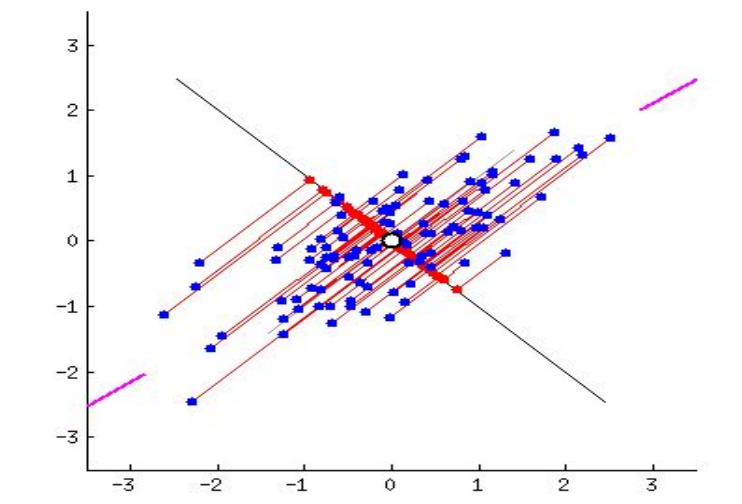
- Must specify K in advance
- Sensitive to initial centroid placement (run multiple times!)
- Assumes spherical clusters of similar size
- Sensitive to outliers (can pull centroids)

Dimensionality reduction

- In medicine, we often measure too many features, and many of them are correlated.
- Too many overlapping features make models:
  - Harder to understand
  - Harder to visualize
  - More prone to noise
  - Slower and less stable
- PCA reduces dozens of features into a few “principal components” that capture the most important patterns.

Principal Component Analysis (PCA)

Dimensionality reduction (1901)



- Reduces many correlated features into few key patterns

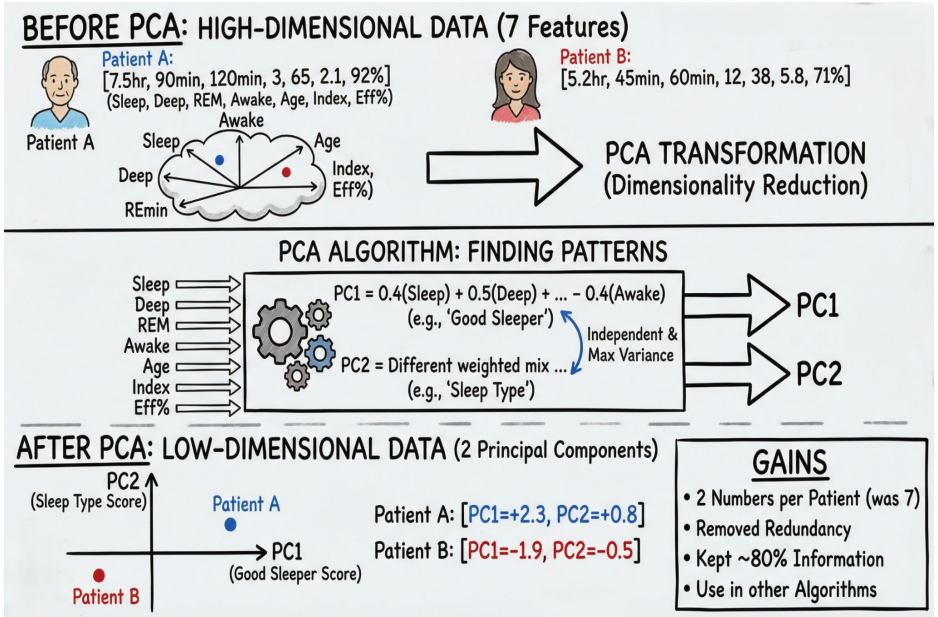
How PCA Works

Features collected from wearable devices:

- Total sleep time (hours)
- Deep sleep (minutes)
- REM sleep (minutes)
- Number of awakenings
- Heart rate variability
- Movement/restlessness score
- Sleep efficiency (%)

The problem:

- These 7 features are highly correlated, they all measure aspects of sleep quality



When to Use PCA

Strengths

- Simplifies high-dimensional clinical data
- Removes redundancy (correlated vitals or labs)
- Reduces noise and improves stability of models
- Makes it easier to visualize patients in 2D
- Helps clustering methods (like K-Means) work better

Weaknesses

- Components can be hard to interpret clinically
- Loses some original detail
- Only captures linear patterns
- Not ideal when absolute feature values matter (e.g., exact lab cutoffs)

Step 5:

Evaluate the model:
How do we know if our model is good?

Why Model Evaluation is Critical in Healthcare?

Healthcare ML has higher stakes than other domains:

- ❏ Wrong prediction → Patient harm
- ❏ Missed diagnosis → Delayed treatment
- ❏ False alarm → Unnecessary procedures, alarm fatigue
- ❏ Biased model → Health disparities

✅ **We need objective evaluation before clinical use**

Key questions we must answer:

- ❏ How accurate is the model?
- ❏ Will it work on new patients in different settings?
- ❏ Can we trust its predictions?

What Can Go Wrong During Training?

Overfitting: Model learns training data too well

Memorizes specific examples instead of patterns

- ❑ Excellent on training data ✓
- ❑ Poor on new patients ✗

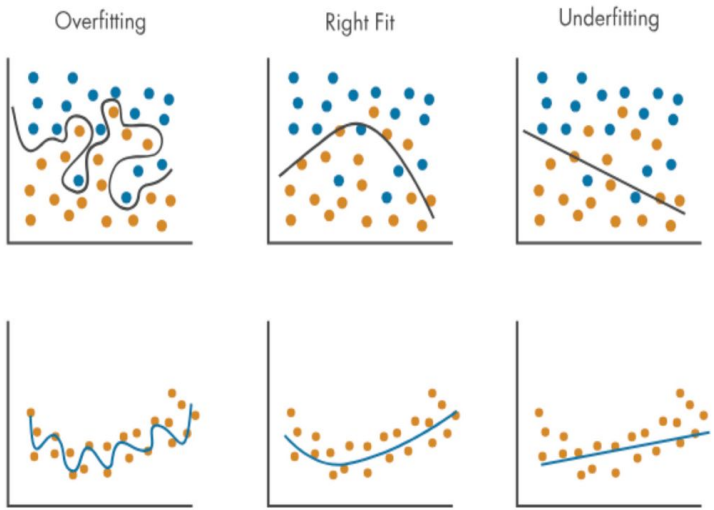
Healthcare risk: Works in your hospital, fails elsewhere

Underfitting: Model too simple to learn patterns

Fails to capture important relationships

- ❑ Poor on training data ✗
- ❑ Poor on new patients ✗

Healthcare risk: Misses critical patterns, doesn't help anyone



Goal: Find the balance

📱 Discuss: Is This Overfitting or Underfitting?

(Talk to your neighbour)

💭 What do you think is happening?

1. Training accuracy 99%, test accuracy 60%.
2. Training accuracy 65%, test accuracy 64%.
3. Training accuracy 90%, test accuracy 88%.

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

How Do We Catch Overfitting? The Train/Test Split

Think of training an ML model like training a medical student:

- Training Set = Practice problems (60-70%)
  - Student learns from 700 practice cases → Model learns patterns from training patients
  - We can check progress, but this doesn't prove mastery
- Test Set = Final exam (30-40%)
  - Student has NEVER seen questions before → Model has NEVER seen patients before
  - This reveals true understanding

🚫 What if the student saw the exam questions? They do great on the exam! But did they learn?

Same problem: If model sees test data during training → falsely high performance

We've trained our model. Now what?

🎯 Does the model make accurate predictions?

- ❑ When it says "sepsis," is the patient actually developing sepsis?

🎯 What types of mistakes does it make?

- ❑ Does it miss dangerous cases?

🎯 Is it better than current practice?

- ❑ Better than clinical judgment alone?

💡 **We need metrics to quantify performance**

The Confusion Matrix

TP = We predicted sepsis, patient has sepsis ✓

TN = We predicted healthy, patient is healthy ✓

FP = False alarm - predicted sepsis, but patient is fine

FN = Missed diagnosis - predicted healthy, but patient has sepsis ⚠️

	Predicted: SEPSIS	Predicted: NO SEPSIS
Actual: SEPSIS	✅ True Positive Caught it!	❌ False Negative MISSED IT (Dangerous!)
Actual: NO SEPSIS	❌ False Positive False alarm	✅ True Negative Correctly healthy

Accuracy: Metric Everyone Knows (Can't Always Trust!)

Accuracy = (TP + TN) / All patients

"How often is the model correct overall?"

✅ When Accuracy Works Well:

Ex: Predicting diabetes in high-risk clinic

- ❏ 400 patients have diabetes
- ❏ 600 patients don't have diabetes
- ❏ Model gets 900/1,000 correct = 90% accurate
- ❏ This actually tells us something useful! ✓

❌ When Accuracy Fails:

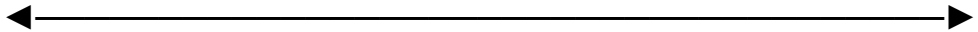
Ex: Cancer screening in general population

- ❏ 10 patients have cancer (1%)
- ❏ 990 patients are healthy (99%)
- ❏ Model predicts "NO CANCER" for everyone
- ❏ Accuracy = 990/1,000 = 99% accurate!
- ❏ But missed ALL 10 cancer cases!

Accuracy works when classes are balanced, but fails when disease is rare

The Two Metrics That Matter Most in Healthcare

Metric	Question It Answers	Formula	Healthcare Goal
Recall (Sensitivity)	"Of all the patients with cancer, how many did we catch?"	TP / (TP + FN)	Catch every case: minimize missed diagnoses
Precision	"When we predict cancer, how often are we right?"	TP / (TP + FP)	Avoid false alarms: minimize unnecessary procedures



High RECALL (Sensitivity)

Catch everything

✓ Few missed cases
✗ Many false alarms

High PRECISION (Confidence)

Only flag when certain

✓ Few false alarms
✗ More missed cases

📱 Precision or Recall? What Matters More?

(Talk to your neighbour)

💭 **For each situation, should we care more about recall or precision?**

- ❏ Case A: Newborn hearing screening
  A quick test flags babies who might have hearing loss and should get a full exam.

- ❏ Case B: Automatic antibiotic stop alert
  An ML tool suggests when to stop antibiotics early to avoid overuse.

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

Other Important Metrics You'll See in Practice

Metric	Question It Answers	Formula	Healthcare Goal
F1-Score	“How do we balance recall & precision?”	Harmonic mean of both	Papers reporting "overall" performance for imbalanced data
AUC-ROC	“Can it separate positive from negative cases?”	0.9+ excellent, 0.7-0.8 decent, 0.5 random	Measuring discrimination ability
MAE (for regression)	“What's the average prediction error?”	Mean Absolute Error in real units	Predicting continuous values (BP, glucose, length of stay)
Calibration	“Do predicted probabilities match reality?”	If model says "30% risk", do 30% actually have disease?	When you need actual risk percentages for decisions
Fairness metrics	“Are results equal across groups?”	Same metrics across demographics	Ensuring no group is disadvantaged

Step 6:

Deployment and Monitoring

→ Train → Test → Evaluate → Deploy → Monitor → Update

🚀 Integration into Clinical Workflow

🧪 Ongoing Performance Monitoring

⚖️ Bias & Safety Audits

🔁 Model Updates & Retraining

👨‍⚕️ Human-in-the-Loop Oversight

📱 ML Case Rally 🚑

(Talk to your neighbour)

💭 How it works:

- ❑ I'll show you a short clinical case.
- ❑ Talk to your neighbour for ~60 seconds.
- ❑ Answer a few questions in Mentimeter.
- ❑ We'll look at the answers together.

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📱 Case 1: The Emergency Department Triage System

💭 **Scenario: An emergency department wants to build an ML model to prioritize patients based on severity.**

Available data:

- ❑ Vital signs (BP, HR, SpO2, temperature)
- ❑ Chief complaint (text)
- ❑ Age, prior visits
- ❑ Lab values (if available)

1. What type of ML task is this? (Classification - urgent/non-urgent/critical)
2. Which features would you include? Which should you exclude and why?
3. What metric matters most: precision or recall? Why?

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📱 Case 2: The Mysterious Cluster

💭 **Scenario: A diabetes clinic has collected data on 1,000 patients (HbA1c, fasting glucose, BMI, age, medication adherence, complication history). They notice patients respond very differently to the same treatment.**

1. You want to discover patient subgroups. What type of learning is this?
2. Which algorithm would you use and why?
3. You try K=3 and K=8 clusters. How do you decide which is better?
4. After clustering, you find one cluster has consistently poor outcomes. What would you do next?
5. Can you use these clusters to predict treatment response for NEW patients? If so, how?

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📱 Case 3: The Fairness Audit

💭 **Scenario: A hospital deployed an ML model to predict no-show appointments. After 6 months, they notice:**
Patients from low-income neighborhoods get flagged 3x more often
The model used: previous no-shows, distance from clinic, age, insurance type

1. Is this model fair? What went wrong?
2. Which features might be problematic and why?
3. If you had to rebuild it, what would you change?
4. What metrics would you track to ensure fairness?

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

📱 Case 4: The Interpretability Challenge

💭 **Scenario: Hospital administrators ask you to choose between:**

Model A: Random Forest, 92% accuracy, but "black box"
Model B: Logistic Regression, 86% accuracy, fully explainable
Use case: Deciding which patients need expensive genetic testing

1. Which would you choose and why?
2. What questions would you ask stakeholders before deciding?
3. Are there ways to make Model A more interpretable?
4. In which medical contexts is interpretability absolutely critical?
5. When might you sacrifice some interpretability for accuracy?

📱 Answer on Mentimeter → www.menti.com | Code: 8309 5558

Wrap up and what's next

🎯 Turned clinical questions into ML problems:
labels, features, supervised vs unsupervised)

🎯Met several classical ML algorithms:
trees & forests, logistic regression, KNN, SVM, clustering, PCA

🎯Saw how to judge a model:
train vs test, overfitting, accuracy vs precision/recall, confusion matrix

🎯The algorithm is only one piece:
how we frame the question, choose data, and evaluate it in the clinical context matters

Next step:

🎯Deep learning models that learn their own features from images, text, and signals, using the same principles you saw today.